

Freedom, Productivity, and Performance  
for Accelerated Computing

# oneAPI Intel® Software Development Tools



All information provided in this deck is subject to change without notice.  
Contact your Intel representative to obtain the latest Intel product specifications and roadmaps.

# Breadth of Intel Developer Software

Comprehensive offering of Intel provided and optimized open-source developer tools including oneAPI products

| Contributions to Open source                                                                                   | Optimized Distributions                                                                                                                    | Optimized Compilers                                                                                                 | Directives                                                                                                                      | Libraries                                                                                                                                                                        | Accelerator Programming                                                         | HW Specific Tools                                                           |
|----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>OpenJDK</li> <li>Python</li> <li>PyTorch</li> <li>TensorFlow</li> </ul> | <ul style="list-style-type: none"> <li>Intel® Distribution of Python</li> <li>Intel® Optimizations for PyTorch &amp; TensorFlow</li> </ul> | <ul style="list-style-type: none"> <li>Intel® oneAPI DPC++/C++ Compiler</li> <li>Intel® Fortran Compiler</li> </ul> | <u>OpenMP 5.x</u> <ul style="list-style-type: none"> <li>OpenMP Parallel</li> <li>OpenMP SIMD</li> <li>OpenMP Target</li> </ul> | <ul style="list-style-type: none"> <li>oneMKL</li> <li>oneDNN</li> <li>oneCCL</li> <li>oneDAL</li> <li>oneTBB</li> <li>oneDPL</li> <li>Intel® IPP</li> <li>Intel® MPI</li> </ul> | <ul style="list-style-type: none"> <li>C++ with SYCL</li> <li>OpenCL</li> </ul> | <ul style="list-style-type: none"> <li>Intrinsics</li> <li>eSIMD</li> </ul> |



Developer Productivity  
Performance Implicit

Developer Control  
Performance Explicit

Developers choose their model of programming  
Intel supports choice and open standards for past, present, and future software

# Analysis & Debug Tools

Get More from Diverse Hardware



## Design

### Intel® Advisor

- Efficiently offload code to GPUs
- Optimize your CPU/GPU code for memory and compute
- Enable more vector parallelism and improve efficiency
- Add effective threading to unthreaded applications



## Debug

### Intel® Distribution for GDB

- Multiple accelerator support with CPU, GPU and FPGA
- Enables deep, system-wide debug of SYCL, C, C++, OpenMP and Fortran cross-architecture applications
- IDE Integration into Microsoft Visual Studio, VS Code and Eclipse



## Tune

### Intel® VTune™ Profiler

- Tune for GPU, CPU, FPGA and NPU\*
- Optimize offload performance
- Supports SYCL, C, C++, Fortran, Python, Go, Java or a mix of languages

\* in technical preview

# Optimize Performance

## Intel® VTune™ Profiler

### Get the Right Data to Find Bottlenecks

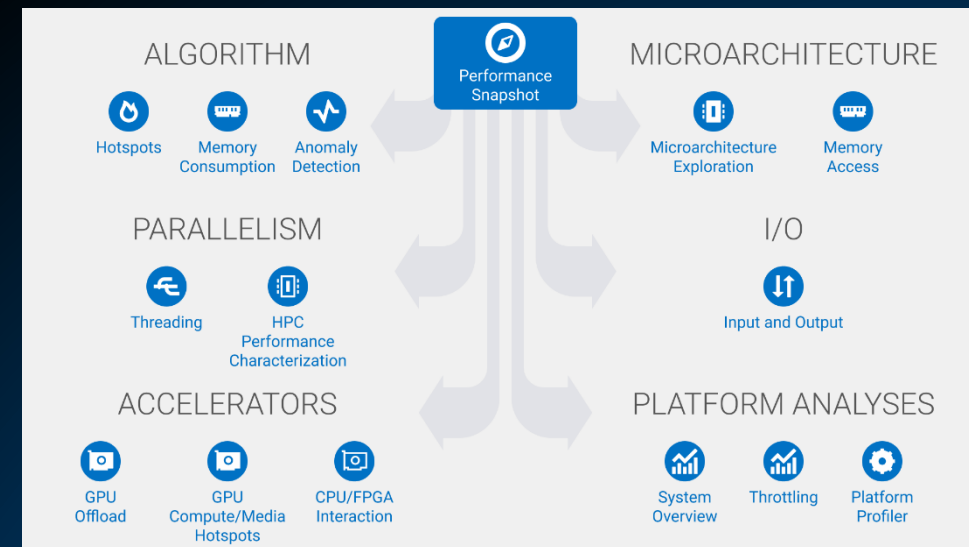
- A suite of profiling for CPU, GPU, NPU, memory, cache, storage, offload, power...
- Application or system-wide analysis
- SYCL, C, C++, Fortran, Python\*, Go\*, Java\*, or a mix
- Linux, Windows, FreeBSD and more
- Containers and VMs

### Analyze Data Faster

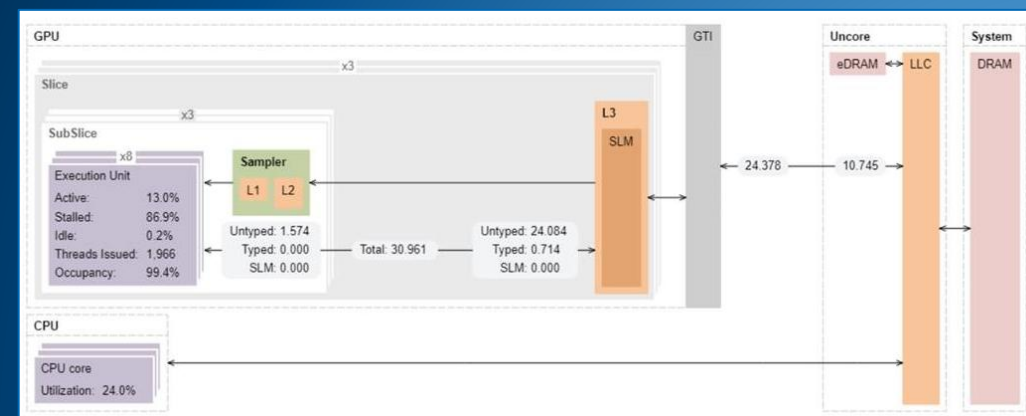
- Collect data HW/SW sampling and tracing w/o re-compilation
- See results on your source, in architecture diagrams, as a histogram, on a timeline...
- Filter and organize data to find answers

### Work Your Way

- User interface or command line
- Profile locally and remotely



| Source |                                    | Assembly |  |  |  |  |  |  |  |  |  |
|--------|------------------------------------|----------|--|--|--|--|--|--|--|--|--|
| Source |                                    |          |  |  |  |  |  |  |  |  |  |
| 158    | dx = ptr[j].pos[0] - ptr[i].pos[0] |          |  |  |  |  |  |  |  |  |  |
| 159    | dy = ptr[j].pos[1] - ptr[i].pos[1] |          |  |  |  |  |  |  |  |  |  |
| 160    | dz = ptr[j].pos[2] - ptr[i].pos[2] |          |  |  |  |  |  |  |  |  |  |



# Rich Set of Profiling Capabilities for Different Problems

Intel® VTune™ Profiler



## Single Thread

Optimize single-threaded performance.



## Multithreaded

Effectively use all available cores.



## System

See a system-level view of application performance.



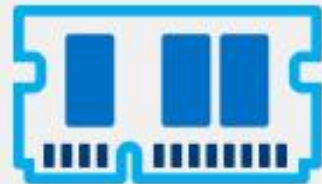
## Media & OpenCL™ Applications

Deliver high-performance image and video processing pipelines.



## HPC & Cloud

Access specialized, in-depth analyses for HPC and cloud computing.



## Memory & Storage Management

Diagnose memory, storage, and data plane bottlenecks.



## Analyze & Filter Data

Mine data for answers.



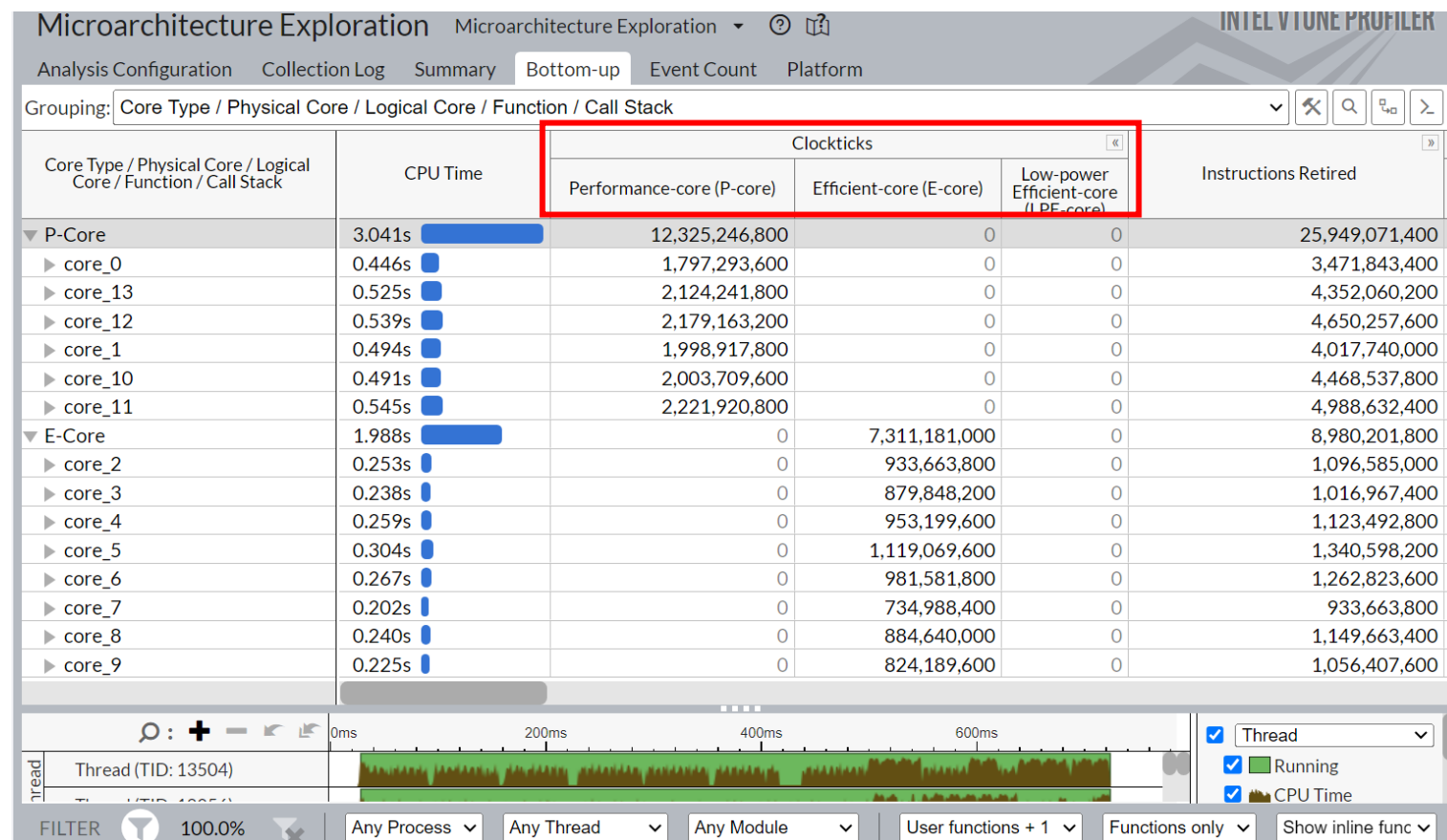
## Environment

Fits your environment and workflow.

## Observe Core Type/Physical core activity using uarch-exploration Analysis

## Observe Core Type/Physical core activity using uarch-exploration Analysis

- CPU Activity for Hybrid design combining Performance (P-cores) and Efficiency (E-cores) cores
  - 6 LNC P-cores
  - 8 SKT E-cores
  - LPE cores (not in figure)
- Multitasking and overall processing efficiency
  - Thread Migration
  - OpenMP region/Barriers



# VTune Suport for iGPU (Intel ARC GPU)

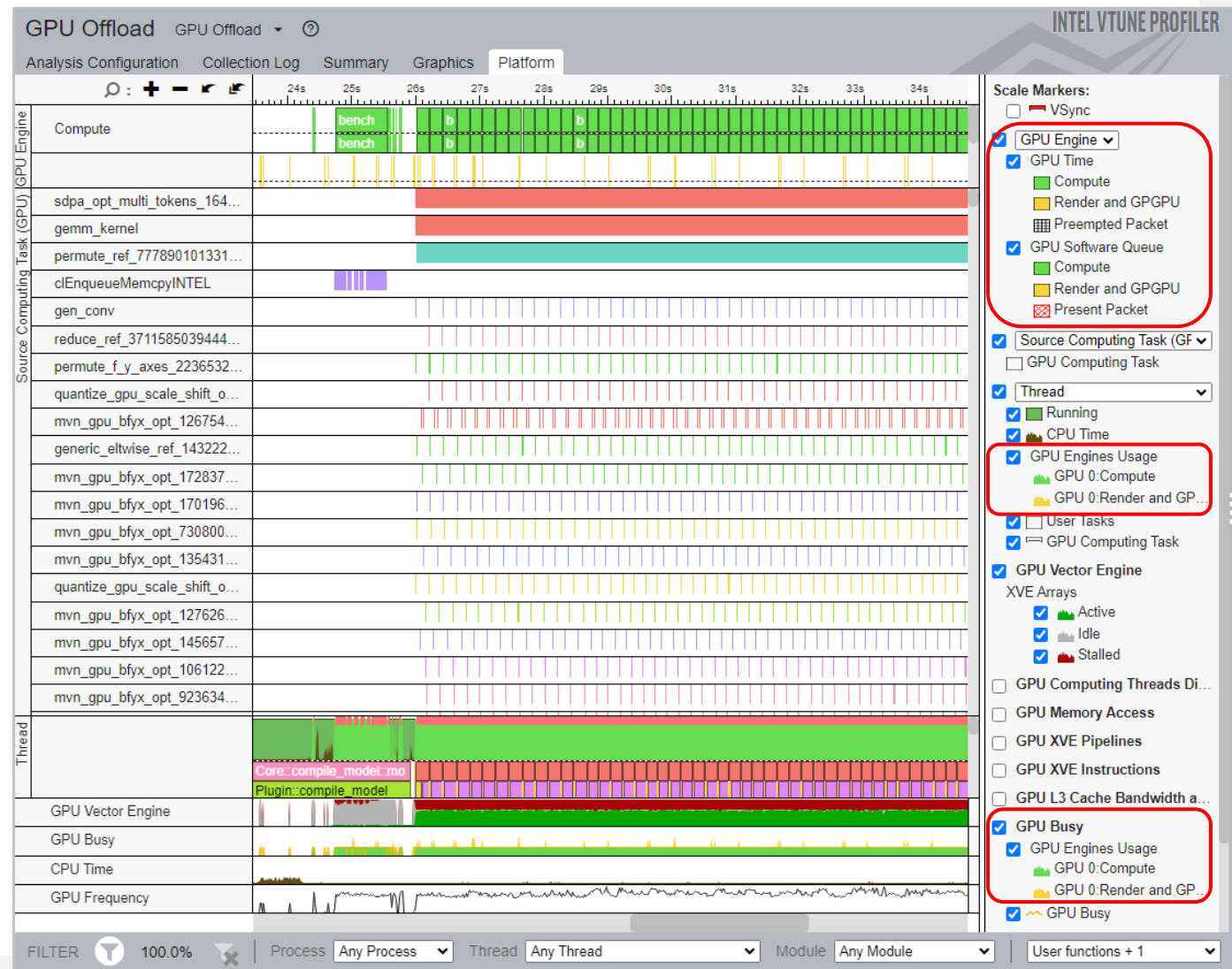
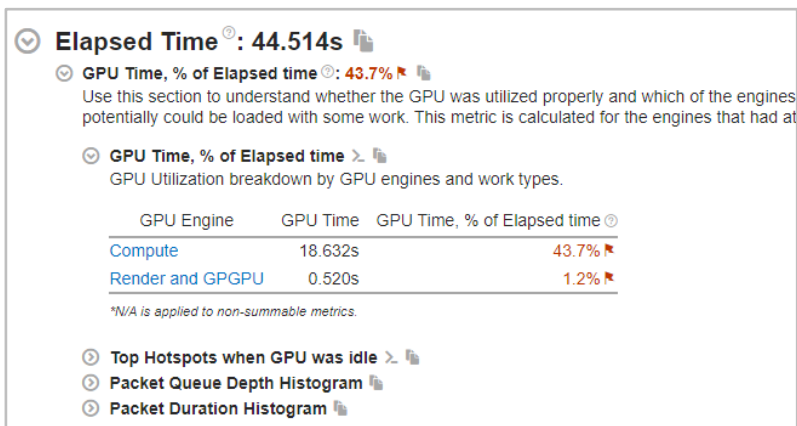
Observe How the Application Utilizes the EUs

## ■ Summary View:

- Utilization Based on Engine Type
- Histogram related to different memory subsystem

## ■ Platform View:

- Compute Tasks and associated Metrics e.g. XVE Utilization, GPU Busy, frequency, CPU Utilization

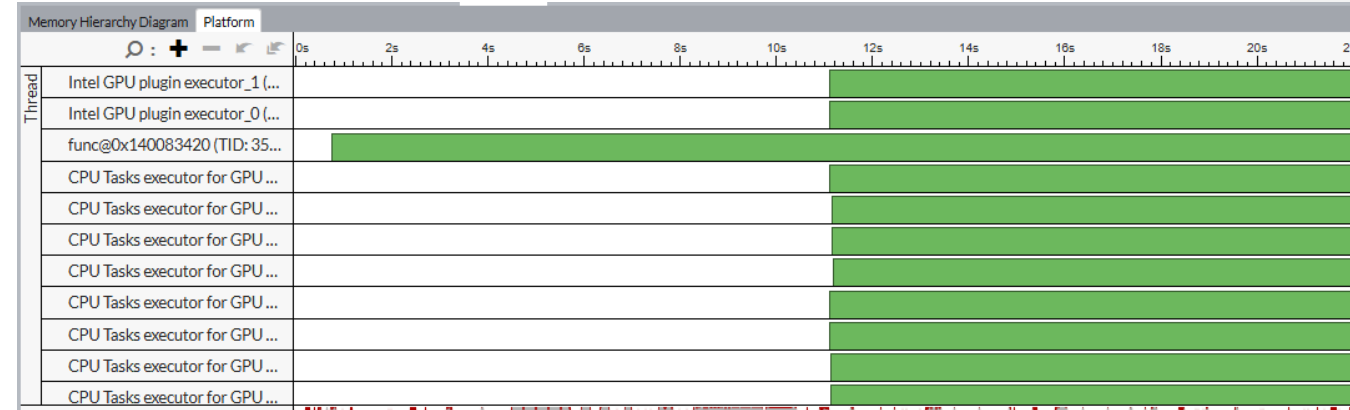




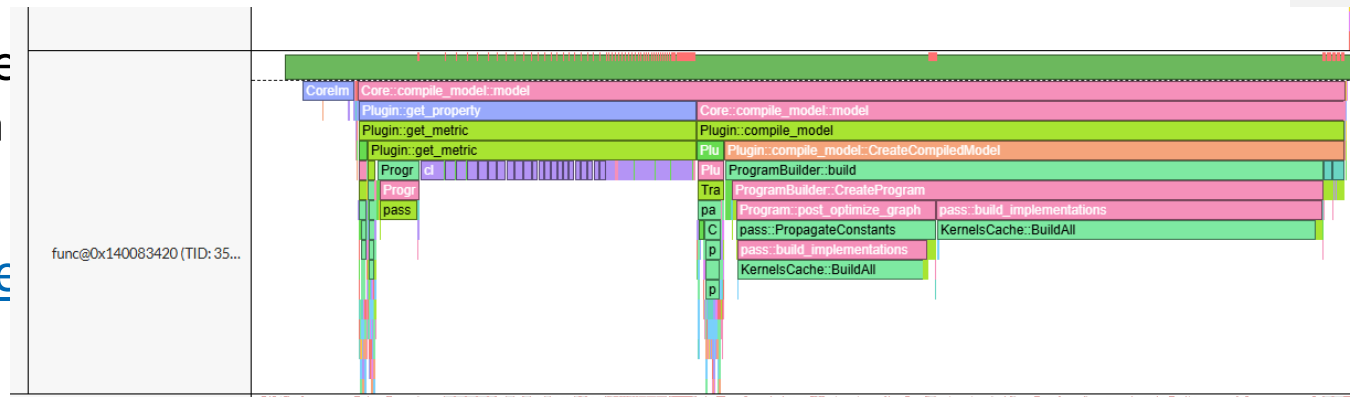
# Profiling OpenVINO Application

- Enable the OpenVINO runtime with Python API
  - `cmake -G "Visual Studio 17 2022" <path\to\OpenVINO™ source code> -DCMAKE_BUILD_TYPE=Release -DENABLE_PROFILING_ITT=ON`
- To identify GPU bottlenecks, run the GPU Compute/Media Hotspots analysis
  - `vtune -c gpu-hotspots -r <path to VTune results directory> -- benchmark_app -m model.xml -d GPU -niter 1000`
- More on detail steps can be found [here](#)

## Before



## After enable ITT





# Intel® oneAPI DPC++/C++ Compiler

## Productive and Performant SYCL Compiler

Uncompromised parallel programming productivity and performance across CPUs and accelerators

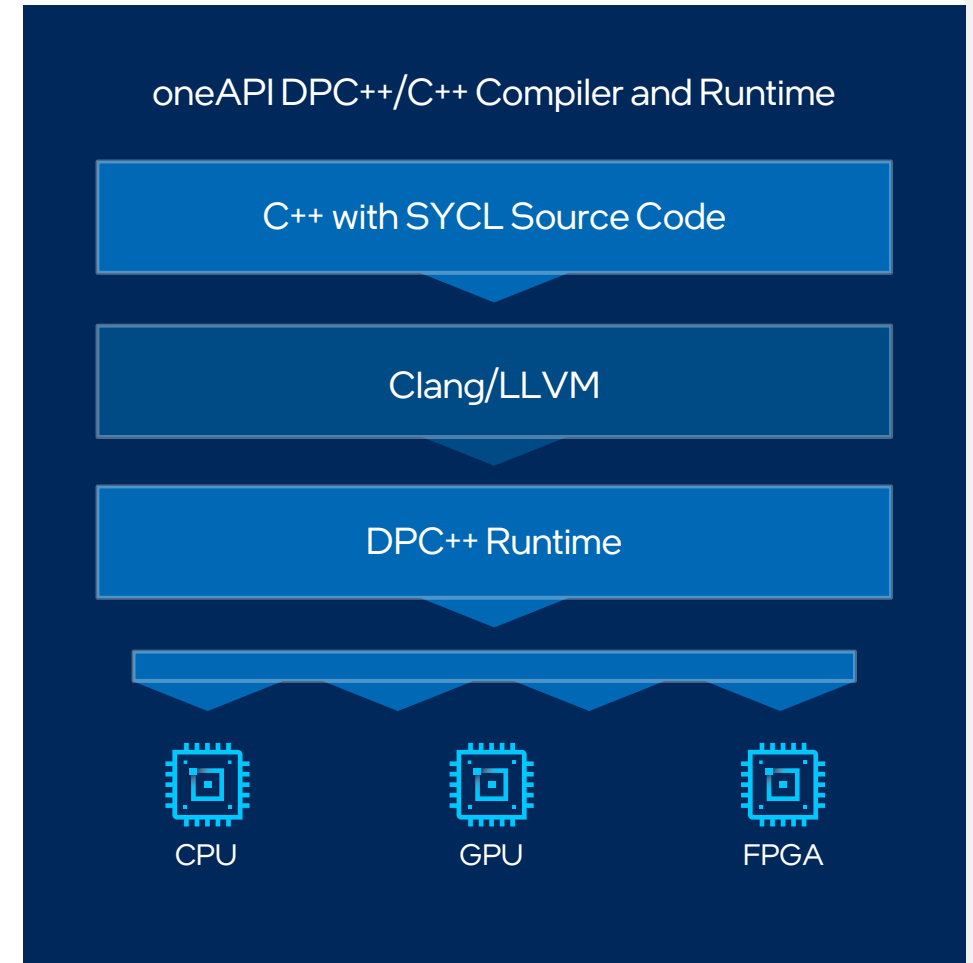
- Allows code reuse across hardware targets, while permitting custom tuning for a specific accelerator
- Open, cross-industry alternative to single architecture proprietary language

### Khronos SYCL Standard

- Delivers C++ productivity benefits, using common and familiar C and C++ constructs
- Created by Khronos Group to support data parallelism and heterogeneous programming

Builds upon Intel's decades of experience in architecture and high-performance compilers

[Learn More & Download](#)



# C++ New Features – *icx*

## What it this?

- Close collaboration with Clang/LLVM community
- *icx* is Clang front-end (FE), LLVM infrastructure
  - PLUS Intel proprietary optimizations and code generation
- Clang FE pulled down frequently from open source, kept current
  - Always up to date in *icx*
  - We contribute! Pushing enhancements to both Clang and LLVM
- Enhancements working with community – better vectorization, opt-report, for example

# Common optimization options

|                                                                                                                         | Linux* icx (icc)                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Disable optimization                                                                                                    | -O0                                                                                                        |
| Optimize for speed (no code size increase)                                                                              | -O1                                                                                                        |
| Optimize for speed (default)                                                                                            | -O2                                                                                                        |
| High-level loop optimization                                                                                            | -O3                                                                                                        |
| Create symbols for debugging                                                                                            | -g                                                                                                         |
| Multi-file inter-procedural optimization                                                                                | -ipo                                                                                                       |
| Profile guided optimization (multi-step build)                                                                          | -fprofile-generate (-prof-gen)<br>-fprofile-use (-prof-use)                                                |
| Optimize for speed across the entire program ("prototype switch")<br><b>fast options definitions changes over time!</b> | -fast same as "-ipo -O3 -static -fp-model fast"<br>(-ipo -O3 -no-prec-div -static -fp-model fast=2 -xHost) |
| OpenMP support                                                                                                          | -fiopenmp (qopenmp)                                                                                        |
| Automatic parallelization                                                                                               | Merged in -O3 -x? (-parallel)                                                                              |

# SIMD: Single Instruction, Multiple Data

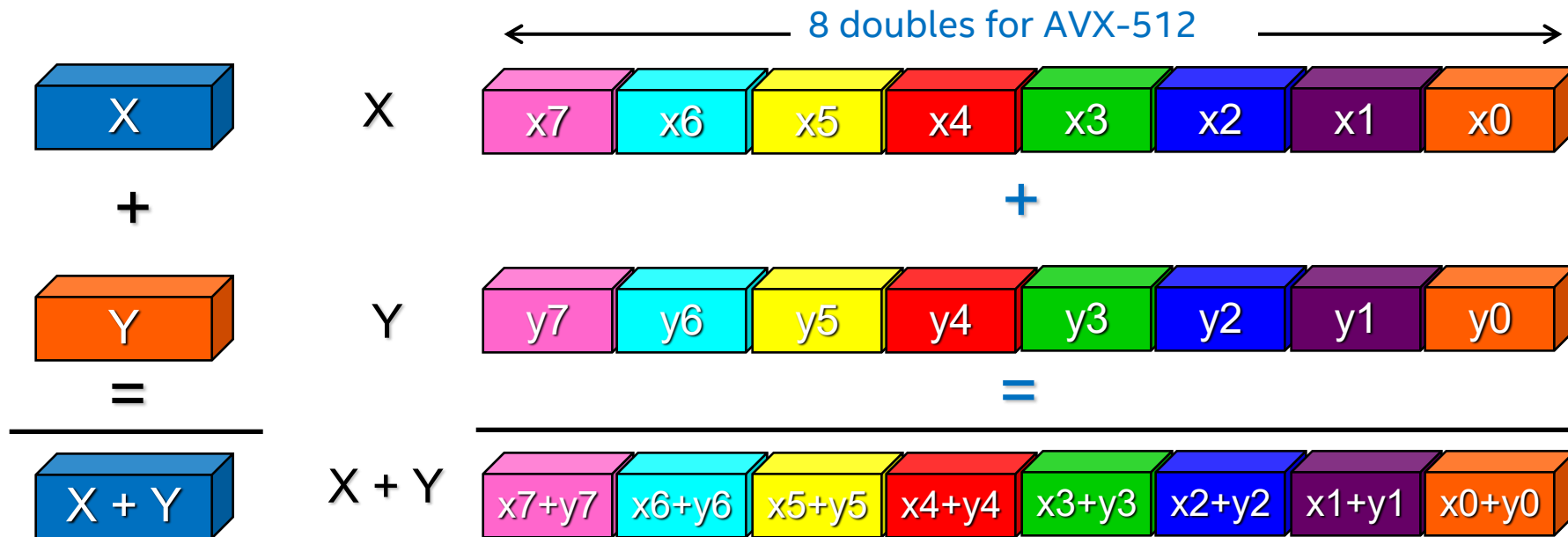
```
for (i=0; i<n; i++) z[i] = x[i] + y[i];
```

## ❑ Scalar mode

- one instruction produces one result
- E.g. `vaddss`, `vaddsd`

## ❑ Vector (SIMD) mode

- one instruction can produce multiple results
- E.g. `vaddps`, `vaddpd`



# Basic Vectorization Switches

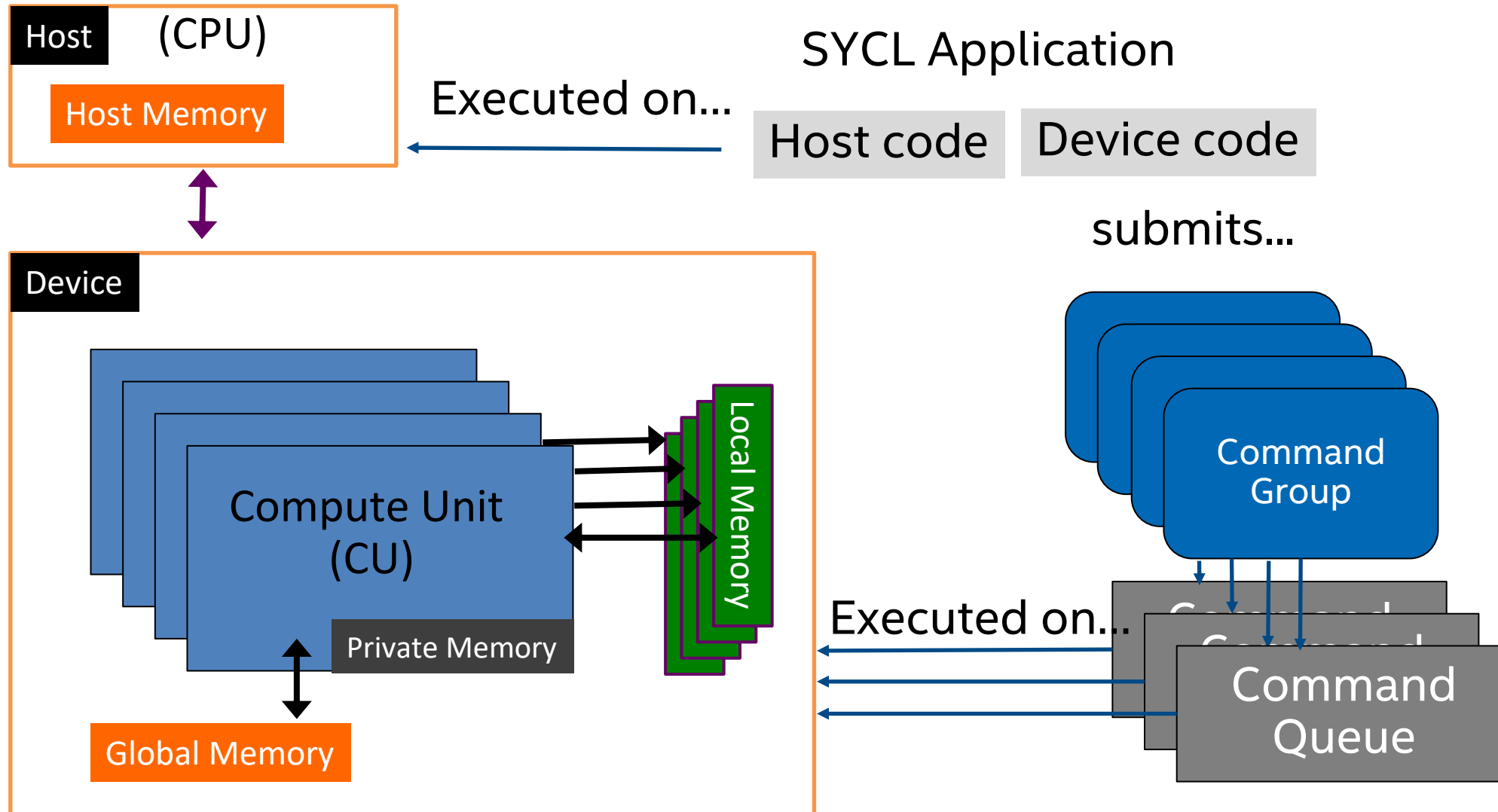
**-x<code> -xHost (Intel proprietary optimizations enabled)**

- Might enable Intel processor specific optimizations
- Processor-check added to “main” routine:  
Application errors in case SIMD feature missing or non-Intel processor with appropriate/informative message

**<code>** indicates a feature set that compiler may target (including instruction sets and optimizations)

- Microarchitecture code names: BROADWELL, HASWELL, SKYLAKE, SKYLAKE-AVX512...
- SIMD extensions: CORE-AVX512, CORE-AVX2, CORE-AVX-I, AVX, SSE4.2, etc.
- Example: `icx -xCORE-AVX2 test.c`

# GPU SYCL Code



# GPU SYCL Code

```
#include <sycl/sycl.hpp>
using namespace sycl;

int main() {
    std::vector<float> A(1024), B(1024), C(1024);
    // some data initialization
    {
        buffer bufA {A}, bufB {B}, bufC {C};
        queue q;
        q.submit([&](handler &h) {
            auto A = bufA.get_access(h, read_only);
            auto B = bufB.get_access(h, read_only);
            auto C = bufC.get_access(h, write_only);
            h.parallel_for(1024, [=](auto i) {
                C[i] = A[i] + B[i];
            });
        });
    }
    for (int i = 0; i < 1024; i++)
        std::cout << "C[" << i << "] = " << C[i] << std::endl;
}
```

Host code

Accelerator  
device code

Host code



# Powerful Performance Libraries

## Rich Functionality

Optimized performance libraries for every use case:

Threading, offload, math, data analytics, data processing, rendering, ray tracing, DNN, comms, crypto, and more.

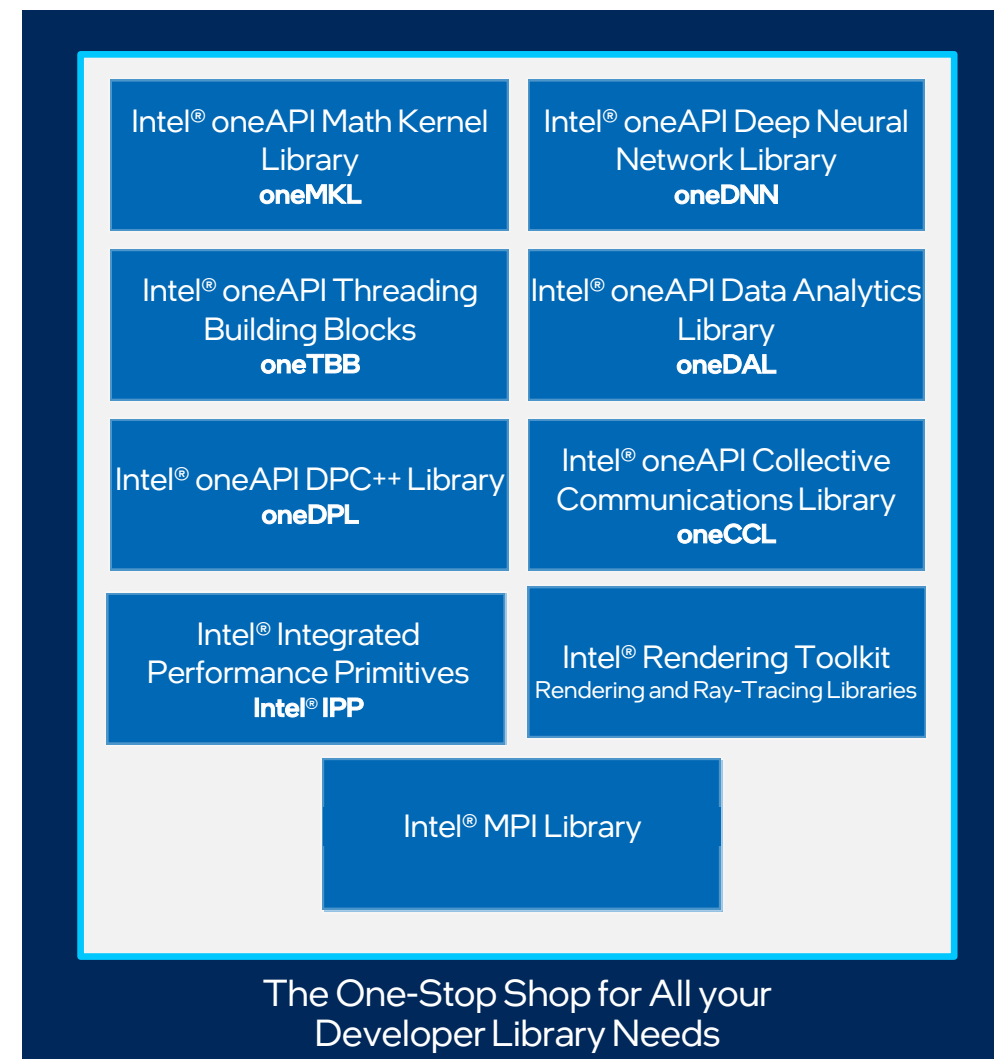
## Realize all the Hardware Value

Designed for acceleration of key domain-specific functions

## Freedom of Choice

Pre-optimized for each target platform for maximum performance

[Explore the Libraries](#)



# Intel® oneAPI Math Kernel Library (oneMKL)

## The Fastest and Most-used Math Library for Intel®-based Systems<sup>1</sup>

Speeds up scientific, engineering, and financial applications by providing highly optimized, threaded, and vectorized math functions

Provides key functionality for dense and sparse linear algebra (BLAS, LAPACK, sparse direct solvers), FFTs, vector math, RNG, summary statistics, interpolation, matrix multiply, and more

Now with latest optimizations for 4<sup>th</sup> Gen Intel® Xeon® Scalable processor and Intel® Data Center GPU Max Series

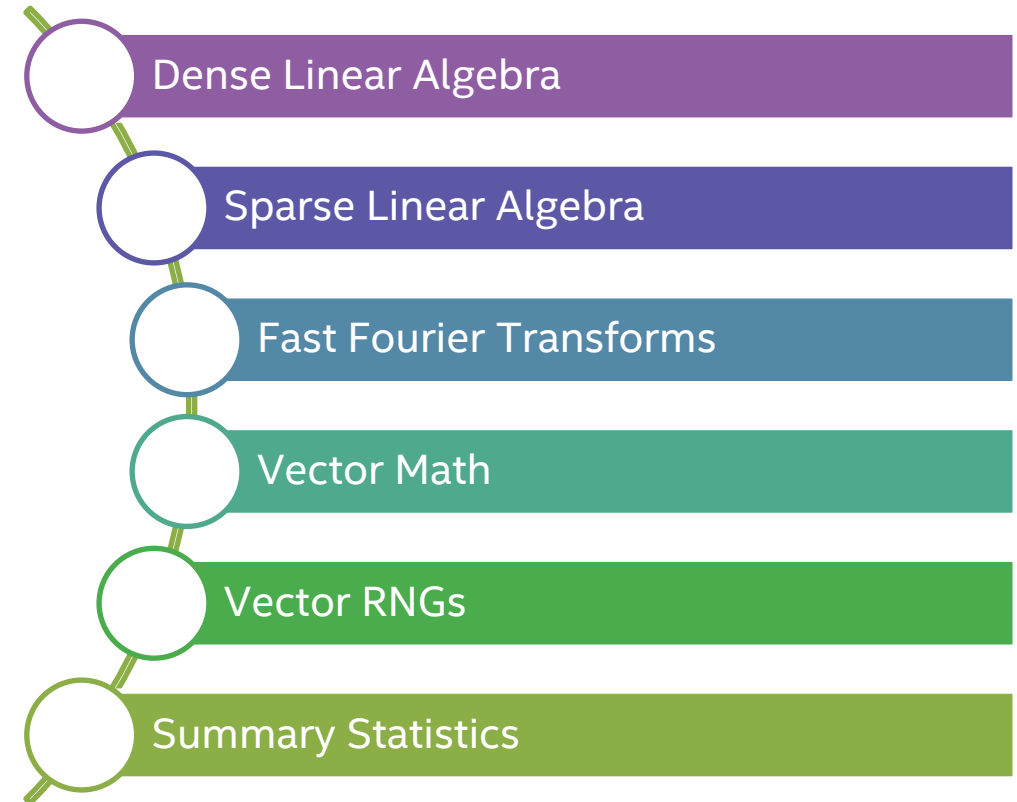
Consistently better performance for current and future generations of Intel® CPUs and GPUs

Dispatches optimized code for each processor automatically without the need to branch code

Language support for SYCL, C and Fortran APIs

Available at no cost and royalty-free.

intel® **oneMKL**



[Learn More & Download](#)

<sup>1</sup> Data from Evans Data Global Development Survey Report 22.1. Performance varies by use, configuration and other factors. Learn more at [www.Intel.com/PerformanceIndex](http://www.Intel.com/PerformanceIndex).

# Intel® Integrated Performance Primitives (Intel® IPP)

## Your Building Blocks for Image, Signal & Data Processing Applications

Ready-to-use, domain-specific functions to accelerate image & signal processing, data compression & cryptography computation tasks

### Image Processing

- Medical Imaging
- Computer Vision
- Digital Surveillance
- ADAS
- Automated Sorting
- Biometric Identification
- Visual Search

### Signal Processing

- Games
- Echo Cancellation
- Telecommunications
- Energy

### Data Compression & Cryptography\*

- Data centers
- ID Verification
- Electronic Signature
- Enterprise data management
- Smart Cards/wallets
- Information Security/Cybersecurity



*\*Effective Q4 2024, Cryptography domain will be a separate product from Intel IPP and will be known as "Intel® Cryptography Primitives Library".*

[Learn More & Download](#)

# Intel® oneAPI Deep Neural Network Library (oneDNN)

## Deliver High Performance Deep Learning

Helps developers create high performance deep learning frameworks

Abstracts out instruction set & other complexities of performance optimizations

Same API for both Intel CPUs and GPUs, use the best technology for the job

Supports Linux, Windows

Open sourced for community contributions



[Learn More & Download](#)

# Intel® Software Developer Tools

## New Toolkits and Sub-bundles

### Intel® oneAPI Base Toolkit

Standards-based, multiarchitecture C++ accelerated computing for CPUs, GPUs, and FPGAs – provides freedom from proprietary software and architecture lock-in

- Intel® oneAPI DPC++/C++ Compiler
- Intel® DPC++ Compatibility Tool
- Intel® Distribution for GDB
- Intel® VTune™ Profiler
- Intel® Advisor
- Intel® oneAPI DPC++ Library
- Intel® oneAPI Threading Building Blocks
- Intel® oneAPI Math Kernel Library
- Intel® oneAPI Deep Neural Network Library
- Intel® oneAPI Data Analytics Library
- Intel® oneAPI Collective Communications Library
- Intel® Integrated Performance Primitives
- Intel® Cryptography Primitives Library
- Optional: FPGA Support Package for the Intel® oneAPI DPC++/C++ Compiler

### Intel® oneAPI HPC Toolkit

Everything developers need for HPC workloads

- Intel® Fortran Compiler
- Intel® MPI Library
- Intel® oneAPI DPC++/C++ Compiler
- Intel® DPC++ Compatibility Tool
- Intel® Distribution for GDB
- Intel® VTune™ Profiler
- Intel® Advisor
- Intel® oneAPI DPC++ Library
- Intel® oneAPI Threading Building Blocks
- Intel® oneAPI Math Kernel Library
- Intel® oneAPI Deep Neural Network Library
- Intel® oneAPI Data Analytics Library
- Intel® oneAPI Collective Communications Library
- Intel® Integrated Performance Primitives
- Intel® Cryptography Primitives Library
- Optional: FPGA Support Package for the Intel® oneAPI DPC++/C++ Compiler

### Intel® Distribution for Python

Drop-in, near-native performance on CPU and GPU for numeric compute, powered by oneAPI

Data Processing and Modeling

- Optimized NumPy, SciPy
- Data Parallel Extensions for NumPy
- oneMKL interfaces

Python Interpreter and Compilers

- Python interpreter
- Numba, Cython
- Data Parallel Extensions for Numba

Advanced Programming Packages

- Data Parallel Control
- tbb4py, mpi4py
- smp

Development Packages & Runtimes

- dal, ipp, mkl, tbb, mpi, openMP, openCL
- icc-rt, fortran-rt, dpcpp-rt

# Resources

## Get Started

- [intel.com/oneapi](https://intel.com/oneapi)
- [Documentation](#) + dev guides
- [Code Samples](#)
- [Intel® Developer Cloud](#)



## Learn

- [Intel Dev Zone Training & Webinars](#)
- [Awesome-oneAPI Repo](#)
- [Summits & Workshops](#): Live & on-demand virtual workshops, community-led sessions
- Videos on [Intel Software YouTube](#) page

